Guía Semana 8: Backend as a Service con Supabase

Oct 6, 2026 · @AntonioJGL

Esta semana conectarás una app Expo a Supabase: registro e inicio de sesión, sesión guardada de forma segura en el teléfono, base de datos remota protegida por usuario y subida de fotos de la cámara a un bucket.

Objetivos y preparación

Al terminar esta guía podrás:

1. Explicar qué resuelve un Backend as a Service y qué piezas de Supabase reemplazan a un backend propio.
2. Implementar registro, inicio y cierre de sesión con correo y contraseña, con rutas protegidas en Expo Router.
3. Guardar la sesión de forma segura en el dispositivo y explicar por qué un token no va en AsyncStorage sin cifrar.
4. Proteger tablas y archivos con Row Level Security para que cada usuario solo acceda a lo suyo.
5. Subir una foto tomada con la cámara a Supabase Storage y mostrarla desde la nube.
6. Manejar credenciales con variables de entorno sin exponerlas en GitHub.

Antes de clase

- [ ] Crear una cuenta y un proyecto en Supabase. Guarda la contraseña de la base de datos en un lugar seguro: no va en el código.
- [ ] Copiar la URL del proyecto y la publishable key (Project Settings, sección API Keys).
- [ ] Leer el tutorial oficial de Supabase con Expo (enlace al final).
- [ ] Tener tu GeoCam de la Semana 7 funcionando, con el trabajo para la casa terminado.

Lo que tu GeoCam debe tener de la Semana 7

Requisito	Por qué se necesita hoy
Columnas `note` y `favorite` en `photos` (tarea C2)	Se suben a la nube junto con la foto

Requisito	Por qué se necesita hoy
Fotos guardadas en la carpeta de documentos (tarea C5)	El archivo debe existir para poder subirlo
Carpeta `drizzle/` intacta y versionada en Git	La migración de hoy se genera sobre ella. Si la borras o la regeneras, la app falla con "table photos already exists"
Proyecto actualizado a SDK 57	`npm install expo@^57.0.0` y luego `npx expo install --fix`

Si no terminaste el trabajo de la Semana 7, pide al docente la solución de referencia y parte de ella.

Ten presente también: las fotos tomadas antes de la tarea C5 pueden no tener ya su archivo, los álbumes siguen siendo solo locales, y borrar una foto todavía la borra solo del teléfono.

Preguntas de repaso

Intenta responderlas antes de clase; las discutiremos al inicio.

1. Si cambias de celular, ¿dónde quedan las fotos de GeoCam?
2. ¿Qué diferencia hay entre autenticación y autorización?
3. ¿Qué es un JWT y qué información lleva?
4. ¿Por qué una llave dentro de una app móvil nunca es realmente secreta?
5. ¿Qué archivo evita que el `.env` llegue a GitHub?

Conceptos clave

La idea central: la llave que va dentro de la app es pública, así que la seguridad no depende de esconderla sino de dos cosas: saber quién es el usuario (autenticación) y que la base de datos decida fila por fila qué puede ver (Row Level Security).

Qué es un Backend as a Service

Un BaaS entrega ya construidas las piezas que casi toda app necesita, para no programar ni administrar un servidor propio. Supabase es un BaaS de código abierto montado sobre PostgreSQL.

Necesidad de la app	Con backend propio	Con Supabase
Guardar datos	Servidor + base de datos + API REST	PostgreSQL con API generada automáticamente
Usuarios y contraseñas	Hash, tokens, recuperación de clave	Supabase Auth
Permisos	Middleware en cada endpoint	Row Level Security dentro de la base
Archivos e imágenes	Servidor de archivos o S3	Supabase Storage (buckets)
Cambios en vivo	WebSockets propios	Supabase Realtime

Usamos Supabase porque es SQL: lo que aprendiste con SQLite y Drizzle en la Semana 7 se transfiere directamente. Firebase (Firestore) es una alternativa válida con modelo de documentos.

Llaves y variables de entorno

Llave	Dónde vive	Qué puede hacer
Publishable key (`sb_publishable_...`, antes "anon key")	Dentro de la app	Solo lo que permitan las políticas RLS
Secret key (`sb_secret_...`, antes "service_role")	Solo en servidores	Se salta RLS: acceso total. Nunca va en la app
Contraseña de la base de datos	Solo en el panel	Conexión directa a PostgreSQL. Nunca va en la app

En Expo, toda variable que empieza por `EXPO_PUBLIC_` queda escrita dentro del bundle de JavaScript. Cualquiera puede extraerla del APK. Por eso:

1. En `.env` solo van la URL y la publishable key.
2. `.env` va en `.gitignore`, y se versiona un `.env.example` con los nombres pero sin valores.
3. La secret key no se pone en el `.env` del proyecto móvil bajo ningún nombre.

```
# .env
EXPO_PUBLIC_SUPABASE_URL=https://xxxxxxxx.supabase.co
EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY=sb_publishable_xxxxxxxx
```

Si una llave llega a GitHub, borrarla en un commit posterior no sirve: queda en el historial. La única solución es rotarla en el panel de Supabase.

Autenticación y sesión

Flujo de inicio de sesión con correo y contraseña:

1. La app envía correo y contraseña a Supabase Auth con `signInWithPassword`.
2. Supabase responde con una sesión: un access token (JWT de vida corta, 1 hora por defecto) y un refresh token (de vida larga).
3. El cliente guarda la sesión en el teléfono para no pedir la clave cada vez que se abre la app.
4. Cada consulta envía el access token; PostgreSQL lo lee con `auth.uid()` para saber quién pregunta.
5. Antes de que el access token venza, el cliente usa el refresh token para pedir uno nuevo (`autoRefreshToken`).
6. `signOut` borra la sesión local e invalida el refresh token.

Dónde guardar la sesión es la decisión de seguridad de la semana:

Opción	Cifrado	Problema
AsyncStorage	No	El refresh token queda en texto plano. Es la opción del tutorial oficial, pero no cumple el requisito del curso
SecureStore directo	Sí (Keychain en iOS, Keystore en Android)	Históricamente iOS rechaza valores mayores a unos 2048 bytes, y la sesión de Supabase suele pesar más
SecureStore por fragmentos	Sí	La sesión se parte en trozos menores al límite. Es la opción del curso

Rutas protegidas: en Expo Router se usa `Stack.Protected` con la propiedad `guard`. Si no hay sesión, las pantallas privadas no existen para el navegador y el usuario cae en el login.

Row Level Security

RLS son reglas que PostgreSQL evalúa en cada fila antes de leerla o modificarla. Sin RLS, cualquier persona con la publishable key puede leer y borrar toda la tabla.

```
alter table posts enable row level security;

-- Cualquier usuario autenticado puede leer
create policy "leer publicaciones" on posts
  for select to authenticated using (true);

-- Solo puedo insertar filas a mi nombre
create policy "crear las mías" on posts
  for insert to authenticated
  with check ((select auth.uid()) = user_id);

-- Solo puedo borrar las mías
create policy "borrar las mías" on posts
  for delete to authenticated
  using ((select auth.uid()) = user_id);
```

`using` filtra las filas existentes (leer, actualizar, borrar). `with check` valida las filas nuevas (insertar, actualizar).

> Una tabla con RLS activo y sin políticas no devuelve nada: ni error ni datos. Si tu consulta llega vacía, revisa primero las políticas.

Storage y tiempo real

Storage guarda archivos en buckets. Un bucket público sirve cualquier archivo a quien tenga la URL; uno privado exige políticas sobre la tabla `storage.objects`. La convención del curso es una carpeta por usuario:

```
-- Ruta de cada archivo: <id del usuario>/<nombre>.jpg
create policy "subir a mi carpeta" on storage.objects
  for insert to authenticated
  with check (
    bucket_id = 'fotos'
    and (storage.foldername(name))[1] = (select auth.uid()::text)
  );
```

Operación de Storage	Permiso necesario en storage.objects
Subir	INSERT

Operación de Storage	Permiso necesario en storage.objects
Reemplazar (upsert)	INSERT, SELECT y UPDATE
Descargar o crear URL firmada	SELECT
Borrar	DELETE

En React Native el archivo se sube como `ArrayBuffer`: `fetch(uri).then(r => r.arrayBuffer())`. Subir un `Blob` o un `FormData` armado a mano produce archivos de 0 bytes.

Realtime envía a la app los cambios de una tabla por WebSocket. La tabla debe agregarse a la publicación `supabase_realtime`, las políticas RLS también aplican a lo que recibes, y el canal se cierra en la limpieza del `useEffect`, igual que las suscripciones de la Semana 6.

Relación con la Semana 7: SQLite sigue siendo la fuente de verdad local. Supabase es el respaldo y el punto de encuentro entre dispositivos. La sincronización automática se construye en la Semana 9.

Ejemplo de clase desde cero: Muro de Clase

Construirás un muro compartido del curso: te registras, publicas un mensaje con foto opcional y ves aparecer las publicaciones de tus compañeros en tiempo real. En clase todos se conectan al proyecto Supabase del docente; para repetirlo por tu cuenta, usa tu propio proyecto y ejecuta el SQL del Paso 3.

El ejemplo cubre las cuatro piezas de la semana en poco código: Auth, tabla con RLS, Storage y Realtime.

Paso 1. Proyecto y dependencias

```
npx create-expo-app@latest muro-de-clase
cd muro-de-clase
npm run reset-project
npx expo install @supabase/supabase-js expo-secure-store expo-image-picker
```

Paso 2. Credenciales

Crea `.env` en la raíz con la URL y la publishable key:

```
EXPO_PUBLIC_SUPABASE_URL=https://xxxxxxxx.supabase.co
EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY=sb_publishable_xxxxxxxx
```

La plantilla de Expo solo ignora `.env*.local`. Agrega `.env` a mano en `.gitignore` y crea `.env.example` con los nombres sin valores:

```
echo ".env" >> .gitignore
git status   # .env NO debe aparecer en la lista
```

Si usas tu propio proyecto para practicar, desactiva la confirmación por correo en la configuración del proveedor Email de Authentication. Así el registro inicia sesión de inmediato. En una app real se deja activa.

Paso 3. Tabla, políticas y bucket

Ejecuta esto en el SQL Editor de Supabase:

```
-- Tabla de publicaciones
create table public.posts (
  id bigint generated always as identity primary key,
  user_id uuid not null default auth.uid() references auth.users (id) on
delete cascade,
  author_email text not null,
  content text not null check (char_length(content) between 1 and 280),
  image_path text,
  created_at timestamptz not null default now()
);

alter table public.posts enable row level security;

create policy "leer publicaciones" on public.posts
  for select to authenticated using (true);

create policy "crear las mías" on public.posts
  for insert to authenticated
  with check ((select auth.uid()) = user_id);

create policy "borrar las mías" on public.posts
  for delete to authenticated
  using ((select auth.uid()) = user_id);

-- Cambios en vivo
alter publication supabase_realtime add table public.posts;

-- Bucket público: cualquiera con la URL ve la imagen, pero solo se sube a la
carpeta propia
insert into storage.buckets (id, name, public) values ('muro', 'muro', true);

create policy "subir a mi carpeta" on storage.objects
```

```
  for insert to authenticated
  with check (bucket_id = 'muro' and (storage.foldername(name))[1] = (select
auth.uid()::text));

create policy "borrar de mi carpeta" on storage.objects
  for delete to authenticated
  using (bucket_id = 'muro' and (storage.foldername(name))[1] = (select
auth.uid()::text));
```

`user_id` tiene `default auth.uid()`: la app no envía el id del usuario, lo pone la base a partir del token. La política de inserción impide que alguien publique a nombre de otro.

Paso 4. Almacenamiento seguro de la sesión

```
// lib/secureStorage.ts
// Adaptador de almacenamiento para Supabase sobre SecureStore.
// La sesión suele superar el límite de ~2048 bytes por valor, así que se
guarda por fragmentos.
import * as SecureStore from 'expo-secure-store';

const CHUNK_SIZE = 1000;
const countKey = (key: string) => `${key}.chunks`;
const chunkKey = (key: string, i: number) => `${key}.${i}`;

async function readCount(key: string) {
  return Number((await SecureStore.getItemAsync(countKey(key))) ?? 0);
}

async function removeItem(key: string) {
  const count = await readCount(key);
  for (let i = 0; i < count; i++) await
SecureStore.deleteItemAsync(chunkKey(key, i));
  await SecureStore.deleteItemAsync(countKey(key));
}

export const secureStorage = {
  async getItem(key: string): Promise<string | null> {
    const count = await readCount(key);
    if (count === 0) return null;
    let value = '';
    for (let i = 0; i < count; i++) {
      const chunk = await SecureStore.getItemAsync(chunkKey(key, i));
      if (chunk === null) return null; // sesión incompleta: se trata como
cerrada
      value += chunk;
    }
```

```
    return value;
  },

  async setItem(key: string, value: string): Promise<void> {
    await removeItem(key);
    const total = Math.ceil(value.length / CHUNK_SIZE);
    for (let i = 0; i < total; i++) {
      await SecureStore.setItemAsync(chunkKey(key, i), value.slice(i *
CHUNK_SIZE, (i + 1) * CHUNK_SIZE));
    }
    await SecureStore.setItemAsync(countKey(key), String(total));
  },

  removeItem,
};
```

Paso 5. Cliente de Supabase

```
// lib/supabase.ts
import { AppState } from 'react-native';
import { createClient } from '@supabase/supabase-js';
import { secureStorage } from './secureStorage';

const url = process.env.EXPO_PUBLIC_SUPABASE_URL;
const key = process.env.EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY;

if (!url || !key) {
  throw new Error('Faltan EXPO_PUBLIC_SUPABASE_URL o
EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY en .env');
}

export const supabase = createClient(url, key, {
  auth: {
    storage: secureStorage,
    autoRefreshToken: true,
    persistSession: true,
    detectSessionInUrl: false, // no hay URL de navegador en una app nativa
  },
});

// El token solo se refresca mientras la app está en primer plano
AppState.addEventListener('change', (state) => {
  if (state === 'active') supabase.auth.startAutoRefresh();
  else supabase.auth.stopAutoRefresh();
});
```

Después de crear o cambiar `.env` reinicia con `npx expo start -c`: las variables se leen al empaquetar.

Paso 6. Contexto de autenticación

```
// context/AuthContext.tsx
import { createContext, useContext, useEffect, useState, type ReactNode }
from 'react';
import type { Session } from '@supabase/supabase-js';
import { supabase } from '@/lib/supabase';

interface AuthValue {
  session: Session | null;
  isLoading: boolean;
  signIn: (email: string, password: string) => Promise<string | null>;
  signUp: (email: string, password: string) => Promise<string | null>;
  signOut: () => Promise<void>;
}

const AuthContext = createContext<AuthValue | null>(null);

export function AuthProvider({ children }: { children: ReactNode }) {
  const [session, setSession] = useState<Session | null>(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    // Se dispara al suscribirse (con la sesión guardada) y en cada login,
logout o refresco
    const { data } = supabase.auth.onAuthStateChange((_event, newSession) =>
{
      setSession(newSession);
      setIsLoading(false);
    });
    return () => data.subscription.unsubscribe();
  }, []);

  const value: AuthValue = {
    session,
    isLoading,
    // Devuelven el mensaje de error para la UI, o null si todo salió bien
    signIn: async (email, password) => {
      const { error } = await supabase.auth.signInWithPassword({ email:
email.trim(), password });
      return error?.message ?? null;
    },
    signUp: async (email, password) => {
```

```
      const { data, error } = await supabase.auth.signUp({ email:
email.trim(), password });
      if (error) return error.message;
      return data.session ? null : 'Revisa tu correo para confirmar la
cuenta.';
    },
    signOut: async () => {
      await supabase.auth.signOut();
    },
  };

  return <AuthContext.Provider value={value}>{children}
</AuthContext.Provider>;
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth debe usarse dentro de AuthProvider');
  return ctx;
}
```

Paso 7. Rutas protegidas y login

```
// app/_layout.tsx
import { ActivityIndicator } from 'react-native';
import { Stack } from 'expo-router';
import { AuthProvider, useAuth } from '@/context/AuthContext';

function RootNavigator() {
  const { session, isLoading } = useAuth();

  if (isLoading) return <ActivityIndicator style={{ flex: 1 }} />;

  return (
    <Stack>
      <Stack.Protected guard={!!session}>
        <Stack.Screen name="index" options={{ title: 'Muro de Clase' }} />
      </Stack.Protected>
      <Stack.Protected guard={!session}>
        <Stack.Screen name="login" options={{ title: 'Ingresar' }} />
      </Stack.Protected>
    </Stack>
  );
}

export default function RootLayout() {
```

```
  return (
    <AuthProvider>
      <RootNavigator />
    </AuthProvider>
  );
}
```

```
// app/login.tsx
import { useState } from 'react';
import { Pressable, StyleSheet, Text, TextInput, View } from 'react-native';
import { useAuth } from '@/context/AuthContext';

export default function LoginScreen() {
  const { signIn, signUp } = useAuth();
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [message, setMessage] = useState<string | null>(null);
  const [busy, setBusy] = useState(false);

  const submit = async (action: typeof signIn) => {
    if (!email.trim() || password.length < 6) {
      return setMessage('Escribe tu correo y una contraseña de al menos 6
caracteres.');
    }
    setBusy(true);
    setMessage(await action(email, password));
    setBusy(false);
  };

  return (
    <View style={styles.container}>
      <TextInput
        style={styles.input}
        placeholder="Correo"
        autoCapitalize="none"
        keyboardType="email-address"
        value={email}
        onChangeText={setEmail}
      />
      <TextInput
        style={styles.input}
        placeholder="Contraseña"
        secureTextEntry
        value={password}
        onChangeText={setPassword}
      />
```

```
      {message && <Text style={styles.message}>{message}</Text>}
      <Pressable style={styles.button} disabled={busy} onPress={() =>
submit(signIn)}>
        <Text style={styles.buttonText}>Iniciar sesión</Text>
      </Pressable>
      <Pressable style={styles.link} disabled={busy} onPress={() =>
submit(signUp)}>
        <Text>Crear cuenta</Text>
      </Pressable>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, padding: 24, gap: 12, justifyContent: 'center',
backgroundColor: '#fff' },
  input: { borderWidth: 1, borderColor: '#ddd', borderRadius: 10, padding:
12, fontSize: 16 },
  message: { color: '#DC2626' },
  button: { backgroundColor: '#111', padding: 14, borderRadius: 10,
alignItems: 'center' },
  buttonText: { color: '#fff', fontWeight: '600' },
  link: { alignItems: 'center', padding: 8 },
});
```

No hay `router.replace` después del login: al cambiar la sesión, `Stack.Protected` cambia de pantalla solo.

Paso 8. Servicio y hook de publicaciones

```
// services/posts.ts
// Única capa que conoce Supabase para las publicaciones (igual que los
repositorios de la Semana 7).
import { supabase } from '@/lib/supabase';

export interface Post {
  id: number;
  user_id: string;
  author_email: string;
  content: string;
  image_path: string | null;
  created_at: string;
}

const BUCKET = 'muro';
```

```
export const imageUrl = (path: string) =>
  supabase.storage.from(BUCKET).getPublicUrl(path).data.publicUrl;

export async function listPosts(): Promise<Post[]> {
  const { data, error } = await supabase
    .from('posts')
    .select('*')
    .order('created_at', { ascending: false })
    .limit(50);
  if (error) throw error;
  return data as Post[];
}

async function uploadImage(userId: string, uri: string): Promise<string> {
  // En React Native el archivo se sube como ArrayBuffer
  const arrayBuffer = await fetch(uri).then((res) => res.arrayBuffer());
  const ext = uri.split('.').pop()?.toLowerCase() ?? 'jpg';
  const path = `${userId}/${Date.now()}.${ext}`; // carpeta = id del usuario
(lo exige la política)
  const { error } = await supabase.storage.from(BUCKET).upload(path,
arrayBuffer, {
    contentType: ext === 'png' ? 'image/png' : 'image/jpeg',
  });
  if (error) throw error;
  return path;
}

export async function createPost(input: {
  userId: string;
  email: string;
  content: string;
  imageUri: string | null;
}) {
  const image_path = input.imageUri ? await uploadImage(input.userId,
input.imageUri) : null;
  const { error } = await supabase
    .from('posts')
    .insert({ author_email: input.email, content: input.content.trim(),
image_path });
  if (error) throw error;
}

export async function deletePost(post: Post) {
  const { error } = await supabase.from('posts').delete().eq('id', post.id);
```

```
  if (error) throw error;
  if (post.image_path) await
supabase.storage.from(BUCKET).remove([post.image_path]);
}
```

```
// hooks/usePosts.ts
import { useCallback, useEffect, useState } from 'react';
import { supabase } from '@/lib/supabase';
import { listPosts, type Post } from '@/services/posts';

export function usePosts() {
  const [posts, setPosts] = useState<Post[]>([]);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  const reload = useCallback(async () => {
    try {
      setPosts(await listPosts());
      setError(null);
    } catch (e) {
      setError(e instanceof Error ? e.message : 'No se pudo cargar el muro');
    } finally {
      setIsLoading(false);
    }
  }, []);

  useEffect(() => {
    reload();

    // Tiempo real: ante cualquier cambio en la tabla, se vuelve a consultar
    const channel = supabase
      .channel('muro-posts')
      .on('postgres_changes', { event: '*', schema: 'public', table: 'posts'
}, () => {
        reload();
      })
      .subscribe();

    return () => {
      supabase.removeChannel(channel); // misma regla de la Semana 6: toda
suscripción se limpia
    };
```

```
  }, [reload]);

  return { posts, isLoading, error, reload };
}
```

Paso 9. Pantalla del muro

```
// app/index.tsx
import { useState } from 'react';
import { Alert, FlatList, Image, Pressable, StyleSheet, Text, TextInput, View
} from 'react-native';
import * as ImagePicker from 'expo-image-picker';
import { useAuth } from '@/context/AuthContext';
import { usePosts } from '@/hooks/usePosts';
import { createPost, deletePost, imageUrl } from '@/services/posts';

export default function MuroScreen() {
  const { session, signOut } = useAuth();
  const { posts, isLoading, error, reload } = usePosts();
  const [content, setContent] = useState('');
  const [imageUri, setImageUri] = useState<string | null>(null);
  const [sending, setSending] = useState(false);

  const user = session!.user; // esta pantalla solo existe con sesión
(Stack.Protected)

  const takePhoto = async () => {
    const permission = await ImagePicker.requestCameraPermissionsAsync();
    if (!permission.granted) return Alert.alert('Se necesita permiso de
cámara');
    const result = await ImagePicker.launchCameraAsync({ mediaTypes:
['images'], quality: 0.5 });
    if (!result.canceled) setImageUri(result.assets[0].uri);
  };

  const publish = async () => {
    if (!content.trim()) return;
    setSending(true);
    try {
      await createPost({ userId: user.id, email: user.email ?? 'anónimo',
content, imageUri });
      setContent('');
      setImageUri(null);
    } catch (e) {
      Alert.alert('No se pudo publicar', e instanceof Error ? e.message :
undefined);
```

```
    } finally {
      setSending(false);
    }
  };

  return (
    <View style={styles.container}>
      <View style={styles.composer}>
        <TextInput
          style={styles.input}
          placeholder="¿Qué está pasando en clase?"
          value={content}
          onChangeText={setContent}
          maxLength={280}
          multiline
        />
        {imageUri && <Image source={{ uri: imageUri }} style={styles.preview}
/>}
        <View style={styles.row}>
          <Pressable onPress={takePhoto}>
            <Text>{imageUri ? 'Cambiar foto' : 'Tomar foto'}</Text>
          </Pressable>
          <Pressable style={styles.button} onPress={publish} disabled=
{sending}>
            <Text style={styles.buttonText}>{sending ? 'Publicando…' :
'Publicar'}</Text>
          </Pressable>
        </View>
      </View>

      {error && <Text style={styles.error}>{error}</Text>}

      <FlatList
        data={posts}
        keyExtractor={(p) => String(p.id)}
        refreshing={isLoading}
        onRefresh={reload}
        ListEmptyComponent={<Text style={styles.empty}>Aún no hay
publicaciones.</Text>}
        renderItem={({ item }) => (
          <Pressable
            style={styles.post}
            onLongPress={() =>
              item.user_id === user.id &&
              Alert.alert('Eliminar publicación', undefined, [
                { text: 'Cancelar', style: 'cancel' },
```

```
                { text: 'Eliminar', style: 'destructive', onPress: () =>
deletePost(item) },
              ])
            }
          >
            <Text style={styles.author}>{item.author_email}</Text>
            <Text>{item.content}</Text>
            {item.image_path && <Image source={{ uri:
imageUrl(item.image_path) }} style={styles.image} />}
            <Text style={styles.date}>{new
Date(item.created_at).toLocaleString('es-CO')}</Text>
          </Pressable>
        )}
      />

      <Pressable style={styles.signOut} onPress={signOut}>
        <Text>Cerrar sesión ({user.email})</Text>
      </Pressable>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#fff' },
  composer: { padding: 16, gap: 8, borderBottomWidth: 1, borderColor: '#eee'
},
  input: { borderWidth: 1, borderColor: '#ddd', borderRadius: 10, padding:
12, minHeight: 60 },
  preview: { width: 80, height: 80, borderRadius: 8 },
  row: { flexDirection: 'row', justifyContent: 'space-between', alignItems:
'center' },
  button: { backgroundColor: '#111', paddingHorizontal: 16, paddingVertical:
10, borderRadius: 10 },
  buttonText: { color: '#fff', fontWeight: '600' },
  error: { color: '#DC2626', padding: 16 },
  empty: { textAlign: 'center', marginTop: 40, color: '#888' },
  post: { padding: 16, gap: 6, borderBottomWidth: 1, borderColor: '#f2f2f2'
},
  author: { fontWeight: '700' },
  image: { width: '100%', aspectRatio: 1, borderRadius: 10 },
  date: { color: '#888', fontSize: 12 },
  signOut: { padding: 16, alignItems: 'center', borderTopWidth: 1,
borderColor: '#eee' },
});
```

Paso 10. Verifica en tu teléfono

- [ ] Regístrate y publica un mensaje con foto: aparece en el feed de todos.
- [ ] Cierra la app por completo y vuelve a abrirla: la sesión sigue iniciada.
- [ ] Intenta borrar la publicación de un compañero: no aparece la opción y, aunque la forzaras, la política lo impide.
- [ ] Cierra sesión: la app vuelve al login sin navegar a mano.

Taller en clase: GeoCam en la nube

En parejas, agrega a tu GeoCam inicio de sesión con Supabase, sesión guardada en SecureStore y un botón para subir cada foto a un bucket privado. Usa tu propio proyecto de Supabase. SQLite sigue siendo la fuente de verdad local: la nube es el respaldo.

Requisitos

- [ ] **T1. Cliente y credenciales.** Instala `@supabase/supabase-js`, `expo-secure-store` y `expo-crypto`, crea `.env`, `.env.example`, agrega `.env` a `.gitignore`, y copia `lib/secureStorage.ts` y `lib/supabase.ts` del ejemplo.
- [ ] **T2. Autenticación.** Crea `context/AuthContext.tsx` y `app/login.tsx` con NativeWind. Protege `(tabs)` y `foto/[id]` con `Stack.Protected`. Agrega un botón de cerrar sesión.
- [ ] **T3. Backend.** Ejecuta el SQL de abajo: tabla `photos` remota con RLS por dueño y bucket privado `geocam` con carpeta por usuario.
- [ ] **T4. Nueva migración local.** Agrega a la tabla `photos` de SQLite cuatro columnas opcionales: `clientId` (UUID único creado en el teléfono), `updatedAt`, `remotePath` y `syncedAt`. Genera la migración con `npm run db:generate` y rellena las fotos antiguas con `ensureSyncFields()`.
- [ ] **T5. Subida.** Crea `services/cloudPhotos.ts` con `uploadPhoto(photo, userId)`: sube el archivo, inserta la fila remota y marca la foto local como sincronizada. Agrega el botón "Subir a la nube" en el detalle de la foto y un ícono de nube en las fotos ya subidas.
- [ ] **T6. Verificación.** Sube una foto y encuéntrala en el panel de Supabase dentro de la carpeta de tu usuario. Cierra la app y comprueba que la sesión sigue iniciada.

SQL para T3

```
create table public.photos (
  id bigint generated always as identity primary key,
  user_id uuid not null default auth.uid() references auth.users (id) on
delete cascade,
  client_id uuid not null unique,      -- UUID creado en el teléfono:
identifica la misma foto en ambos lados
  storage_path text not null unique,
  latitude double precision,
  longitude double precision,
  note text,
  favorite boolean not null default false,
  source text not null check (source in ('camera', 'gallery')),
  taken_at timestamptz not null,
  updated_at timestamptz not null,     -- última modificación hecha por el
usuario (base de la Semana 9)
  created_at timestamptz not null default now()
);

alter table public.photos enable row level security;

-- Cada usuario solo ve, crea y borra sus propias filas
create policy "ver las mías" on public.photos
  for select to authenticated using ((select auth.uid()) = user_id);

create policy "crear las mías" on public.photos
  for insert to authenticated with check ((select auth.uid()) = user_id);

create policy "borrar las mías" on public.photos
  for delete to authenticated using ((select auth.uid()) = user_id);

-- Bucket privado: sin política de lectura nadie puede ver los archivos
insert into storage.buckets (id, name, public) values ('geocam', 'geocam',
false);

create policy "subir a mi carpeta" on storage.objects
  for insert to authenticated
  with check (bucket_id = 'geocam' and (storage.foldername(name))[1] =
(select auth.uid()::text));

create policy "leer mi carpeta" on storage.objects
  for select to authenticated
  using (bucket_id = 'geocam' and (storage.foldername(name))[1] = (select
auth.uid()::text));
```

```
create policy "borrar de mi carpeta" on storage.objects
  for delete to authenticated
  using (bucket_id = 'geocam' and (storage.foldername(name))[1] = (select
auth.uid()::text));
```

Diferencia con el ejemplo: el muro usa un bucket público porque todos ven todo. GeoCam usa uno privado porque las fotos con ubicación son datos personales.

Código de apoyo

Layout raíz: primero las migraciones locales, después la autenticación.

```
// app/_layout.tsx (fragmento)
function RootNavigator() {
  const { session, isLoading } = useAuth();
  if (isLoading) return <View className="flex-1 bg-neutral-950" />;

  return (
    <Stack screenOptions={{ headerShown: false }}>
      <Stack.Protected guard={!!session}>
        <Stack.Screen name="(tabs)" />
        <Stack.Screen name="foto/[id]" options={{ headerShown: true, title:
'Foto' }} />
      </Stack.Protected>
      <Stack.Protected guard={!session}>
        <Stack.Screen name="login" />
      </Stack.Protected>
    </Stack>
  );
}

export default function RootLayout() {
  const { success, error } = useMigrations(db, migrations);
  // ... pantallas de error y carga de la Semana 7 ...
  return (
    <AuthProvider>
      <RootNavigator />
    </AuthProvider>
  );
}
```

`app/index.tsx` queda fuera de los dos grupos y redirige según la sesión: `<Redirect href={session ? '/geocam' : '/login'} />`.

Columnas nuevas para T4. Las cuatro son opcionales porque SQLite no permite agregar una columna obligatoria sin valor constante a una tabla que ya tiene filas:

```
// db/schema.ts (dentro de la tabla photos)
import * as Crypto from 'expo-crypto';

  // Identidad compartida con la nube: el id autoincremental solo vale en
este teléfono
  clientId: text('client_id').unique().$defaultFn(() => Crypto.randomUUID()),
  // Última modificación hecha por el usuario
  updatedAt: integer('updated_at', { mode: 'timestamp_ms' }).$defaultFn(() =>
new Date()),
  // Ruta del archivo en el bucket y momento de la última sincronización
  remotePath: text('remote_path'),
  syncedAt: integer('synced_at', { mode: 'timestamp_ms' }),
```

Las fotos tomadas antes de la migración quedan con `clientId` y `updatedAt` vacíos. Se rellenan una sola vez al arrancar, después de las migraciones:

```
// db/repositories/photos.ts (método nuevo)
  async ensureSyncFields() {
    const rows = await
db.select().from(photos).where(isNull(photos.clientId));
    for (const row of rows) {
      await db
        .update(photos)
        .set({ clientId: Crypto.randomUUID(), updatedAt: row.updatedAt ??
row.createdAt })
        .where(eq(photos.id, row.id));
    }
  },
```

Regla desde hoy: cada vez que el usuario edita una foto (nota, favorita, álbum), `photosRepo.update` también escribe `updatedAt: new Date()`. Una foto está pendiente de subir cuando `syncedAt` está vacío o es anterior a `updatedAt`.

Subida de una foto:

```
// services/cloudPhotos.ts
import { supabase } from '@/lib/supabase';
import { photosRepo } from '@/db/repositories/photos';
import type { Photo } from '@/db/schema';

const BUCKET = 'geocam';

export async function uploadPhoto(photo: Photo, userId: string) {
  if (!photo.clientId) throw new Error('La foto no tiene clientId: ejecuta
ensureSyncFields()');
```

```
  // 1. Archivo local -> ArrayBuffer
  const arrayBuffer = await fetch(photo.uri).then((res) =>
res.arrayBuffer());

  // 2. Subir a la carpeta del usuario (lo exige la política del bucket).
  //    El nombre es el clientId: la misma foto siempre va a la misma ruta.
  const path = `${userId}/${photo.clientId}.jpg`;
  const { error: uploadError } = await supabase.storage
    .from(BUCKET)
    .upload(path, arrayBuffer, { contentType: 'image/jpeg' });
  if (uploadError) throw uploadError;

  // 3. Fila remota con los metadatos
  const { error: insertError } = await supabase.from('photos').insert({
    client_id: photo.clientId,
    storage_path: path,
    latitude: photo.latitude,
    longitude: photo.longitude,
    note: photo.note,
    favorite: photo.favorite,
    source: photo.source,
    taken_at: photo.createdAt.toISOString(),
    updated_at: (photo.updatedAt ?? photo.createdAt).toISOString(),
  });
  if (insertError) throw insertError;

  // 4. Marcar la foto local como sincronizada (sin tocar updatedAt)
  await photosRepo.markSynced(photo.id, path);
}

/** URL temporal para ver un archivo de un bucket privado. */
export async function signedUrl(path: string) {
  const { data, error } = await
supabase.storage.from(BUCKET).createSignedUrl(path, 3600);
  if (error) throw error;
  return data.signedUrl;
}
```

Pistas

- Si la subida responde "new row violates row-level security policy", la ruta no empieza por el id del usuario o falta la política de `insert` del bucket.
- Si la consulta devuelve una lista vacía sin error, falta la política de `select` o no hay sesión iniciada.

- Si `process.env.EXPO_PUBLIC_SUPABASE_URL` llega `undefined`, falta reiniciar con `npx expo start -c` o el archivo no se llama exactamente `.env`.
- `photosRepo.markSynced(id, path)` escribe solo `remotePath` y `syncedAt`. No debe cambiar `updatedAt`, o la foto volvería a quedar pendiente.
- Subir dos veces la misma foto falla con "duplicate key" o "The resource already exists". Es lo esperado: `client_id` es único. El trabajo para la casa lo maneja.
- Si el archivo queda en 0 bytes, se subió un `Blob` o `FormData`. Debe ser `ArrayBuffer`.
- Si la app muestra "table photos already exists", generaste la migración sin la carpeta `drizzle/` original.
- Hoy el orden es subir archivo, luego fila. Si falla la fila, queda un archivo huérfano en el bucket. El trabajo para la casa lo resuelve.

Trabajo para la casa

En parejas, convierte la subida manual del taller en un respaldo confiable: sincronización de pendientes, borrado coherente, consulta de la nube y datos locales separados por usuario. Entrega antes de la clase de la Semana 9.

Tareas

- [ ] **C1. Subir pendientes.** Botón "Sincronizar" que sube todas las fotos con `syncedAt` vacío, una por una, mostrando el avance ("3 de 7"). Si una falla, las demás continúan y al final se informa cuántas quedaron pendientes.
- [ ] **C2. Subida sin duplicados ni huérfanos.** Repetir la sincronización no debe crear filas ni archivos repetidos. Si el archivo sube pero la fila falla, el archivo se borra del bucket antes de reportar el error.
- [ ] **C3. Borrado coherente.** Al eliminar una foto sincronizada, se borran también su fila remota y su archivo del bucket. Sin conexión, la app avisa y no borra nada a medias.
- [ ] **C4. Pestaña Nube.** Nueva pestaña que lista las fotos del usuario guardadas en Supabase, con imágenes cargadas mediante URL firmada y actualización en tiempo real. Debe funcionar al iniciar sesión en otro teléfono.
- [ ] **C5. Datos locales por usuario.** Hoy, si otra persona inicia sesión en el mismo teléfono, ve las fotos locales de la anterior. Agrega con una migración la columna `ownerId` a la tabla local y filtra todas las consultas por el usuario de la sesión.
- [ ] **C6. Auditoría de credenciales.** Demuestra en el README que `.env` nunca entró al historial (`git log --all --full-history -- .env` sin resultados), que existe `.env.example` y que en el código no aparece ninguna secret key.

- [ ] **C7. Documentación.** README con el diagrama de las dos bases (SQLite local y PostgreSQL remoto), el SQL completo de tablas y políticas, un GIF del flujo y el `AI-LOG.md` de la semana.

Reto opcional: OAuth o enlace mágico

Agrega un segundo método de inicio de sesión: proveedor OAuth (GitHub es el más rápido de configurar) o enlace mágico por correo. Requiere `expo-auth-session`, `expo-web-browser` y `expo-linking`, definir un `scheme` en `app.json` y registrar la URL de retorno en la lista de redirecciones permitidas de Supabase. En Expo Go la URL de retorno empieza por `exp://`, distinta de la del build final; documenta en el AI-LOG cómo lo resolviste.

Preguntas para sustentar

1. ¿Por qué la publishable key puede estar en el código y la secret key no?
2. Si alguien extrae la publishable key del APK, ¿qué puede hacer con ella en tu proyecto?
3. ¿Qué pasa con la sesión si el usuario pasa 30 días sin abrir la app?
4. ¿Por qué la foto se sube antes que la fila, y qué problema crea ese orden?

Entregable semanal

Entrega el enlace a tu repositorio de GitHub con el flujo de autenticación funcional, la sesión en SecureStore y la carga de imágenes a Storage: el taller en clase más el trabajo para la casa, sin credenciales en el historial.

Antes de entregar, verifica

- [ ] Registro, inicio y cierre de sesión funcionan y muestran los errores al usuario.
- [ ] Sin sesión no se puede entrar a ninguna pantalla privada.
- [ ] La sesión sigue iniciada después de cerrar la app por completo.
- [ ] La sesión se guarda en SecureStore, no en AsyncStorage.
- [ ] RLS está activo en la tabla y en el bucket: con una segunda cuenta no ves ni borras fotos ajenas.
- [ ] Las fotos subidas no pesan 0 bytes y quedan en la carpeta de tu usuario.
- [ ] `.env` está en `.gitignore`, existe `.env.example` y no hay ninguna secret key en el código.
- [ ] La carpeta `drizzle/` está versionada con la migración nueva.
- [ ] `README.md` y `AI-LOG.md` están en la raíz.

Una secret key o una contraseña de base de datos en el repositorio es una falta grave, aunque la hayas borrado en un commit posterior. Si te pasa, rótala en el panel de Supabase y avisa al docente.

Errores de IA que debes detectar

La IA genera	Problema	Corrección
`SUPABASE_SERVICE_ROLE_KEY` o una secret key en la app	Se salta RLS: acceso total a la base desde cualquier APK	Solo la publishable key; lo demás vive en servidores
`process.env.SUPABASE_URL` sin prefijo	En Expo llega `undefined`	`EXPO_PUBLIC_SUPABASE_URL` y reiniciar con `-c`
`storage: AsyncStorage`	El refresh token queda sin cifrar	Adaptador sobre SecureStore
`storage: SecureStore` directo	Falla o advierte con sesiones mayores a ~2048 bytes	Adaptador por fragmentos
`detectSessionInUrl: true`	No existe URL de navegador en una app nativa	`false`
`supabase.auth.session()` o `supabase.auth.user()`	API de supabase-js v1, eliminada	`getSession()`, `getUser()` y `onAuthStateChange`
`router.replace('/home')` después del login	Navegación manual que se desincroniza con la sesión	`Stack.Protected` con `guard`
Subir con `FormData` o `Blob`	Archivos de 0 bytes en React Native	`ArrayBuffer` con `fetch(uri).then(r => r.arrayBuffer())`
Políticas `using (true)` para insertar, actualizar o borrar	Cualquier usuario modifica datos ajenos	Comparar `auth.uid()` con `user_id`
"Desactiva RLS para que funcione"	Deja la tabla abierta a todo internet	Escribir la política que falta

La IA genera	Problema	Corrección
Canal de Realtime sin `removeChannel`	Conexiones abiertas y eventos duplicados	Limpiar en el `useEffect`
`@supabase/auth-helpers-react` u otros paquetes de Next.js	No aplican a React Native	Solo `@supabase/supabase-js`

Ejemplo de AI-LOG.md

```
# Registro de Auditoría de IA (AI-LOG)

**Estudiante(s):** Nombre Apellido, Nombre Apellido
**Semana:** 8
**Proyecto:** GeoCam en la nube – Taller Integrador 2

## 1. Prompts Utilizados
- "Configura el cliente de Supabase en Expo con TypeScript para que la sesión
  se guarde de forma segura con expo-secure-store."

## 2. Código Generado vs. Código Modificado
- **¿Qué generó la IA?:** Un cliente con storage: AsyncStorage y un
comentario
  diciendo que SecureStore "no es compatible" con Supabase.
- **¿Qué modifiqué/corregí?:** Escribí un adaptador que parte la sesión en
  fragmentos de 1000 caracteres dentro de SecureStore, porque el problema
real
  es el límite de tamaño por valor y no una incompatibilidad.

## 3. Alucinaciones o Errores Detectados
- Al fallar la subida por RLS, la IA propuso ejecutar
  "alter table photos disable row level security". Eso dejaba la tabla
abierta
  a cualquiera con la publishable key. El error real era que la ruta del
  archivo no empezaba por el id del usuario; corregí la ruta y mantuve RLS.
```

Documentación de referencia

- Supabase con Expo React Native (tutorial oficial)
- Guía de Expo para Supabase
- Expo SecureStore
- Autenticación en Expo Router
- Control de acceso en Supabase Storage

- Deep linking para OAuth en apps nativas

Para la próxima semana: lee la introducción de TanStack Query y llega con las columnas `clientId`, `updatedAt`, `remotePath` y `syncedAt` funcionando. Piensa en esta pregunta: ¿cómo lograr que la app suba sola lo pendiente cuando vuelve la conexión y baje lo que se creó en otro teléfono?